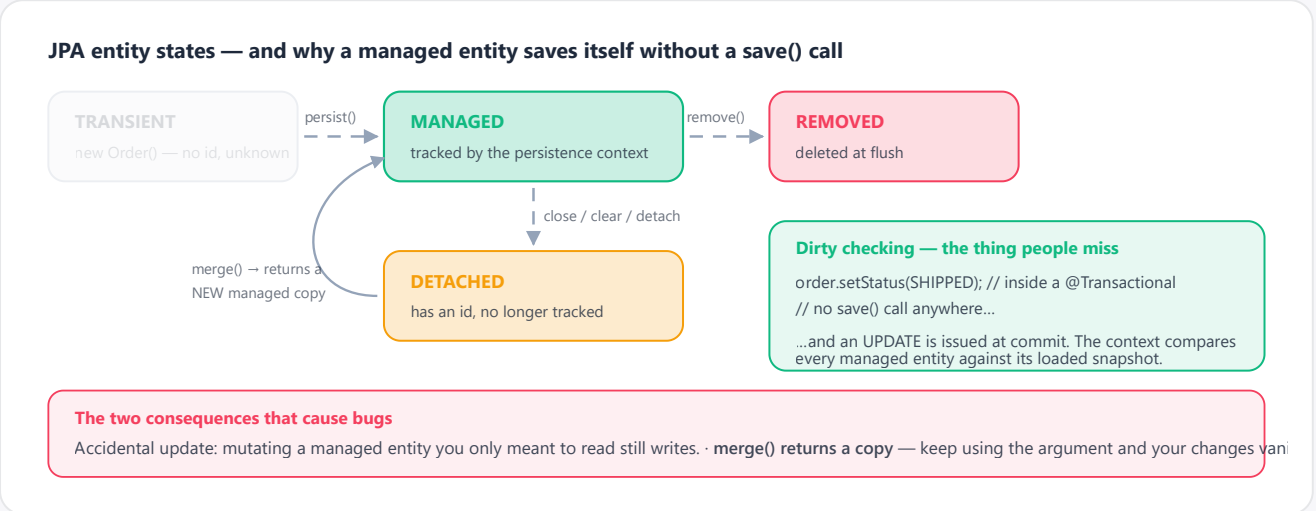
The Persistence Context and Entity States

Why it matters

The persistence context is the thing that makes JPA feel magical and behave surprisingly. Updates happen without a save call; the same query returns the identical object twice; changes you did not intend get written. All three follow from one concept, and candidates who know it can predict JPA's behaviour instead of memorising its symptoms.

The concept, simply

The persistence context is a **first-level cache** scoped to a transaction. It tracks every entity it has loaded or persisted, guarantees one object per database row within its scope, and compares each tracked entity against a loaded snapshot at flush time — **dirty checking**.



The managed state is where the surprises live: an entity in it is watched, so any mutation becomes an UPDATE at commit whether or not you asked for one.

State	Meaning
Transient	`new Order()` — no id, unknown to JPA
Managed	Tracked; changes are written automatically at flush
Detached	Has an id but is no longer tracked — the transaction ended
Removed	Scheduled for deletion at flush

Dirty checking is the behaviour to internalise: inside a transaction, mutating a managed entity issues an `UPDATE` at commit with no `save()` anywhere. That is powerful, and it is also why an "innocent" setter call in a read path can write to the database.

The first-level cache guarantees identity. Loading the same row twice in one transaction returns the *same object*, and the second load issues no query. This is not a performance cache you configure — it is a correctness guarantee, and it is always on.

Code examples

JAVA

Copy

```java
@Transactional
public void ship(Long orderId) {
    Order order = repository.findById(orderId).orElseThrow();
    order.setStatus(SHIPPED);
    // no save() — an UPDATE is issued at commit by dirty checking
}

// Identity within a transaction:
@Transactional
public void demo(Long id) {
    Order a = repository.findById(id).orElseThrow();
    Order b = repository.findById(id).orElseThrow();    // no second query
    System.out.println(a == b);                         // true — same obje
}
```

```
    Order o = repository.findById(id).orElseThrow();
    o.setLastViewedAt(Instant.now());      // ✗ writes to the DB at commit
    return OrderView.from(o);
}
// ✓ Mark read paths explicitly — it also lets Hibernate skip the snapshot
//   and dirty-check entirely, which is a real performance win on big read
@Transactional(readOnly = true)
```

The two errors everyone meets:

JAVA

Copy

```
// 1. LazyInitializationException — the context closed with the transactio
//    so the proxy has nothing to load from.
public OrderResponse get(Long id) {           // ✗ no @Transactional
    Order o = repository.findById(id).orElseThrow();
    return new OrderResponse(o.getCustomer().getName());   // ✗ throws
}
// ✓ Fixes, best first: fetch what you need in the query (JOIN FETCH or
//   @EntityGraph), or return a projection. Extending the transaction to t
//   web layer works and is worse — it holds a connection for the whole re
// ✗ Never "fix" it with spring.jpa.open-in-view=true. Boot even warns abc
//   it at startup: it keeps the context open for the entire request, hide
//   N+1 problems, and holds a database connection while rendering.

// 2. merge() returns a COPY.
@Transactional
public void update(Order detached) {
    repository.save(detached);          // returns a managed copy
    detached.setStatus(SHIPPED);        // ✗ mutating the DETACHED one — lc
}
@Transactional
public void update(Order detached) {
    Order managed = repository.save(detached);
    managed.setStatus(SHIPPED);         // ✓ mutate what save() returned
}
```

Flush is not commit. Hibernate flushes — issues the SQL — before a query that might be affected by pending changes, and at commit. So `persist()` does not necessarily execute an `INSERT` immediately; statements are batched and ordered by Hibernate, which is why the SQL you see in the log can appear in an unexpected order.

The second-level cache is a different thing entirely and a common confusion. The first-level cache is per transaction, mandatory and always on. The second-level cache is per `SessionFactory`, shared across transactions, optional, and needs a provider such as Ehcache or Hazelcast. Only the first is guaranteed by the specification.

`readOnly = true` **earns its keep.** Beyond preventing accidental writes, Hibernate skips taking the dirty-checking snapshot, which halves the memory footprint of a large read and removes the comparison at flush. On a query returning thousands of rows the difference is measurable, which makes it a genuine optimisation rather than documentation.

Common mistakes

- **Mutating a managed entity in a read path.** It writes at commit. Use `readOnly = true`.
- **Touching a lazy association outside the transaction.** `LazyInitializationException`.
- **Enabling `open-in-view` to make it go away.** Hides N+1 and holds a connection all request.
- **Using the argument after `merge()` / `save()`.** The managed instance is the returned one.
- **Confusing first- and second-level caches.** One is per transaction and mandatory; the other is shared and optional.
- **Assuming `persist()` executes immediately.** Statements are flushed later and reordered.
- **Loading thousands of entities to update them.** A bulk `@Modifying` query is orders of magnitude cheaper.

Q: "What is the persistence context?"

A: A transaction-scoped first-level cache that tracks every entity it has loaded or persisted. It guarantees one object per row within its scope, so loading the same row twice returns the identical instance with no second query, and it dirty-checks tracked entities at flush — comparing them to a loaded snapshot and issuing updates for anything that changed.

Q: "Why does an entity update without calling `save()`?"

A: Dirty checking. Inside a transaction the entity is managed, so at commit Hibernate compares it to the snapshot taken at load and issues an `UPDATE` for the changed columns. The flip side is that mutating an entity you only meant to read also writes — which is one reason to mark read paths `@Transactional(readOnly = true)`.

Q: "What causes `LazyInitializationException`?"

A: Touching a lazy association after the persistence context has closed — typically in the controller or during serialisation, once the transactional service method has returned. The right fix is to fetch what you need inside the transaction with a fetch join or an entity graph, or to return a projection. Turning on `open-in-view` makes the symptom disappear while holding a database connection for the whole request and hiding N+1 problems.

Q: "What is the difference between `persist` and `merge`?"

A: `persist` makes a transient entity managed and expects it to be new. `merge` takes a detached entity, copies its state onto a managed instance, and **returns that instance** — the argument stays detached. Forgetting the return value is a classic bug: you keep mutating the detached object and nothing is saved.

A: The first-level cache is the persistence context: per transaction, mandatory, always on, and it guarantees object identity. The second-level cache is per `SessionFactory`, shared across transactions and sessions, optional, and requires a provider like Ehcache. Only the first is part of the guarantees you can rely on by default.

Q: "What does `readOnly = true` actually do?"
A: It sets the flush mode to manual so nothing is written, and Hibernate skips taking the dirty-checking snapshot. That second part is a real optimisation — on a query loading thousands of entities it roughly halves the memory used and removes the comparison work at flush. It also documents intent, which prevents the accidental-write bug.

Cheat-card

- Persistence context = **transaction-scoped first-level cache** + identity guarantee + **dirty checking**.
- States: **transient → managed → detached / removed**.
- Managed entity mutated ⇒ **UPDATE at commit, no save() needed**.
- Read paths: `@Transactional(readOnly = true)` — no accidental writes, no snapshot.
- `merge()` **returns the managed copy** — use the return value.
- `LazyInitializationException` = context closed → fetch join / entity graph / projection. **Not** `open-in-view`.
- Flush ≠ commit; statements are batched and reordered.
- L1 = per transaction, mandatory. L2 = shared, optional, needs a provider.

QUICK CHECK — no marks, just for you.

1. Inside a transactional method you load an entity and call a setter, with no save() anywhere. What happens at commit?

- ◯ Nothing — without save() no SQL is issued
- ◯ An UPDATE is issued — the entity is managed, and dirty checking compares it against the snapshot taken at load
- ◯ The change is discarded when the context closes
- ◯ An exception, because the entity was not explicitly saved

Check answer

2. What causes LazyInitializationException, and what is the right fix?

- ◯ Touching a lazy association; fix it by making the association EAGER
- ◯ A null association; fix it by initialising the field
- ◯ Touching a lazy association after the persistence context has closed; fix it by fetching what you need inside the transaction with a fetch join, an entity graph, or a projection
- ◯ A missing @Transactional on the repository; add one there

Check answer

3. What does merge() return, and why does it matter?

- ◯ A boolean indicating whether the entity existed
- ◯ void — it modifies the argument in place
- ◯ The same instance you passed in

A MANAGED COPY — the argument stays detached, so mutating the argument after merging changes nothing

Check answer

Previous
Entities, Mapping and Repositories

Next
Lazy Loading and the N+1 Problem

LEARN MORE

ARTICLES & DOCS

- Data access and transactions — Spring Framework reference
  docs.spring.io
- Hibernate ORM user guide
  hibernate.org

VIDEOS



Transaction Management in hibernate in spring boot Interview questions | with Example | Code Decode
Code Decode

Hand-picked references for this lesson.